

Topic : C#

1] What is Common Language Runtime (CLR)?

- CLR handles program execution for various languages including C#.
- The architecture of CLR handles memory management, garbage collection, security handling.

2] Extension Methods in C#?

- Extension methods are a powerful feature in C# that allows you to add new functionality to existing types without modifying their source code.
- They can be extremely useful in making your code more readable and maintainable.

3] What is garbage collection in C#?

- Garbage collection is the process of freeing up memory that is captured by unwanted objects.
- When you create a class object, automatically some memory space is allocated to the object in the heap memory.
- Now, after you perform all the actions on the object, the memory space occupied by the object becomes waste.
- It is necessary to free up memory.

4] What are the types of classes in C#?

- **Static class:** Static class, defined by the keyword 'static' does not allow inheritance. Therefore, you cannot create an object for a static class.
- **Partial class:** Partial class, defined by the keyword 'partial' allows its members to partially divide or share source (.cs) files.
- **Abstract class:** Abstract classes are classes that cannot be instantiated where you cannot create objects. Abstract classes work on the OOPS concept of abstraction. Abstraction helps to extract essential details and hide the unessential ones.

- **Sealed class:** Sealed classes are classes that cannot be inherited. Use the keyword sealed to restrict access to users to inherit that class.

5] What is a managed and unmanaged code?

Managed code:

- Managed code runs on the managed runtime environment than the operating system itself.
- It Provides various services like a garbage collector, exception handling, etc.

Unmanaged code:

- When the code doesn't run on CLR, it is an unmanaged code that works outside the .NET framework.
- They don't provide services of the high-level languages and therefore, run without them.

6] Explain Pass by value & Pass by reference?

Pass by Value

- Pass a copy of the original value to the function rather than reference.
- It does not modify the original value.
- The changes made to the parameter inside of the called method will not have an effect on the original value.
- The Variable value is directly stored in the memory.

Pass by Reference

- It passes a reference of arguments to the function.
- The changes in passed values are permanent and modify the original variable values.
- The Reference types won't store variable values directly in the memory.
- Rather, it will store the memory address of the variable value to indicate where the value is being stored.

7] What is Recursion in C#?

- Function calls itself again and again.

- Reduce the size of code.
- It Provide Reusability.

8] What is OOP?

- OOP stands for Object-Oriented Programming.
- Object-oriented programming is about creating objects that contain both data and methods.
- OOP is faster and easier to execute
- OOP provides a clear structure for the programs
- OOP helps to keep the C# code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
- OOP makes it possible to create full reusable applications with less code and shorter development time

9] Explain Access Modifiers in OOP?

- **private**

The code is only accessible within the same class.

- **public**

The code is accessible for all classes.

- **protected**

The code is accessible within a class or in a class that is inherited from that class.

- **internal**

The code is accessible within it's own assembly but not from another assembly.

10] What is a constructor in C#?

- A special method of the class that is automatically invoked when an instance of the class is created is called a constructor.
- The main use of constructors is to initialize the private fields of the class while creating an instance for the class.
- When you have not created a constructor in the class, the compiler will automatically create a default constructor of the class.
- A class can have any number of constructors.

- A constructor doesn't have any return type, not even void.
- A static constructor can not be a parametrized constructor.
- Within a class, you can create one static constructor only.

Types of constructor in C#:

1. Default Constructor
2. Parameterized Constructor
3. Copy Constructor
4. Static Constructor
5. Private Constructor

- **Default Constructor:**

A constructor without any parameters is called a default constructor.

- **Parameterized Constructor:**

A constructor with at least one parameter is called a parameterized constructor.

- **Copy Constructor:**

The constructor which creates an object by copying variables from another object is called a copy constructor.

- **Static Constructor:**

When a constructor is created using a static keyword, it will be invoked only once for all of the instances of the class and it is invoked during the creation of the first instance of the class or the first reference to a static member in the class.

- **Private Constructor:**

When a constructor is created with a private specifier, it is not possible for other classes to derive from this class, neither is it possible to create an instance of this class. They are usually used in classes that contain static members only.

11] What is Polymorphism? Explain Compiler & Runtime Polymorphism.

- Polymorphism means "many forms", and it occurs when we have many classes that are related to each other by inheritance.

- It uses those methods to perform different tasks.
- This allows us to perform a single action in different ways.

Method Overloading

- Method Overloading is a type of polymorphism. It has several names like "Compile Time Polymorphism" or "Static Polymorphism," and sometimes it is called "Early Binding".
- It means creating multiple methods in a class with the same names but different signatures.
- The compiler automatically calls the required method to check the number of parameters and their type passed into that method.

Method Overriding

- Method Overriding is a type of polymorphism. It has several names like "Run Time Polymorphism" or "Dynamic Polymorphism," and sometimes it is called "Late Binding".
- Method Overriding means having two methods with the same name and same signatures.
- one should be in the base class, and another method should be in a derived class [child class].
- You can override the functionality of a base class method to create the same name method with the same signature in a derived class.
- You can achieve method overriding using inheritance.
- Virtual and Override keywords are used to achieve method overriding.

12] What is Inheritance? Explain types of Inheritance in C#.

- It is possible to inherit fields and methods from one class to another.
- It used (:) symbol to extends Parent class into Child class.

Types of Inheritance:

1] Single Inheritance:

- It Contains only 2 class i.e Parent & Child.
- It derived Parent class into Child class.
- Means can access all properties & methods from Parents to Child class.

2] Multilevel Inheritance:

- When one class inherits another class which is further inherited by another class, it is known as multilevel inheritance in C#.
- Inheritance is transitive so the last derived class acquires all the members of all its base classes.

3] Hierarchical Inheritance:

- It Contains Single Parent but can have multiple childs.
- Each child can access properties & methods from Parent class.

13] How we can access Parent data member into child class?

By using 'base' keyword.

Ex.

```
class A
```

```
{
```

```
    public void disA()
    {
        //code
    }
}
```

```
class B : A
{
    public void disB()
    {
        base.disA();
        //code
    }
}
```

14] Explain Sealed Keyword?

- We can avoid increment if we apply sealed keyword on variable

Ex.

```
sealed int a=1; a++;
```

- We can avoid method overriding by using sealed keyword. Ex.

```
class A
```

```
{  
    sealed public virtual void dis()  
    {  
        //code  
    }  
}
```

```
class B : A
```

```
{  
    public override void dis()  
    {  
        //code  
    }  
}
```

- We can avoid inheritance also. Ex.

```
sealed class A
```

```
{  
    methods;  
}
```



```
class B : A
{
    methods;
}
```

15] Explain static class Keyword?

- o C# static class contains only static members.
- o C# static class cannot be instantiated.
- o C# static class is sealed.
- o C# static class cannot contain instance constructors.

16] How we can achieve Abstraction in C#?

Abstraction in C# is the process to hide the internal details and showing functionality only.

method which is declared abstract and has no body is called abstract method. It can be declared inside the abstract class only. Its implementation must be provided by derived classes.

There are 2 ways to achieve abstraction:

1] Interface

2] Abstract class

Interface

- It is a blueprint of a class.
- It is like abstract class because all the methods which are declared inside the interface are abstract methods.
- It cannot have method body and cannot be instantiated.
- It is used *to achieve multiple inheritance* which can't be achieved by class.
- It is used *to achieve fully abstraction* because it cannot have method body.
- Its implementation must be provided by class.

- The class which implements the interface, must provide the implementation of all the methods declared inside the interface.

Syntax:

```
interface interface_name  
{  
    method declaration;  
}
```

Ex.

```
interface Test  
{  
    void dis();  
}  
  
class TestImpl : Test  
{  
    public void dis()  
    {  
        Console.WriteLine("This is Interface method");  
    }  
    static void Main(string []args)  
    {
```

```
        TestImpl obj=new TestImpl();
        obj.dis();
    }
}
```

Abstract Class

- abstract class is a class which is declared abstract.
- It can have abstract and non-abstract methods.
- It cannot be instantiated.
- Its implementation must be provided by derived classes.
- Here, derived class is forced to provide the implementation of all the abstract methods.

Syntax:

```
abstract class classname
{
    //abstract or non-abstract methods;
}
```

Ex.

```
abstract class Test
{
    abstract void
    dis1(); public void
    dis2()
    {
        CW("It is Non-abstract method");
    }
}
```

```

}
class TestImpl : Test
{
    public void dis1()
    {
        CW("this method belongs from Test class");
    }
    static void main(string []args)
    {
        TestImpl obj=new TestImpl();
        obj.dis1(); //abstract method
        calling
        obj.dis2(); //non-abstract method calling
    }
}

```

17] Explain Encapsulation in C#.

- Encapsulation is the concept of wrapping data into a single unit.
- It collects data members and member functions into a single unit called class.
- The purpose of encapsulation is to prevent alteration of data from outside. This data can only be accessed by getter functions of the class.
- A fully encapsulated class has getter and setter functions that are used to read and write data.
- This class does not allow data access directly.

Ex.

```

class Emp
{

```

```

        public int Id {get; set;}
        public string Name {get; set;}
        public double Salary {get; set;}
    }
    class Impl
    {
        static void Main(string []args)
        {
            Emp obj=new Emp();
            //setting values to the
            properties obj.Id=101;
            obj.Name="Anil";
            obj.Salary=1000;
            //getting values
            Console.WriteLine("ID is "+obj.Id);
            Console.WriteLine("Name is
            "+obj.Name);
            Console.WriteLine("Salary is
            "+obj.Salary);
        }
    }
}

```

18] Explain Exception Handling in C#?

It is *a process to handle runtime errors*. We perform exception handling so that normal flow of the application can be maintained even after runtime errors.

Exception Blocks:

1] try : It is used to place the code that may throw exception.

2] catch : is used to handled the exception.

3] throw : is used to raise custom exception.

4] finally : It is used to execute important code which is to be executed whether exception is handled or not.

Ex.

class Test

```
{
    static void Main(string[] args)
    {
        try
        {
            int a = 10;
            int b = 0;
            int x = a /
            b;
        }
        catch (Exception e)
        {
            Console.WriteLine(e);
        }
        finally
        {
            Console.WriteLine("Finally block is executed");
        }
        Console.WriteLine("Rest of the code");
    }
}
```

19] Explain Delegates?

- A delegate is a type that represents references to methods with a particular parameter list and return type.
- When you instantiate a delegate, you can associate its instance with any method with a compatible signature and return type.
- You can invoke (or call) the method through the delegate instance.
- A delegate can be declared using delegate keyword.

20] Explain Generics?

- Generic is a concept that allows us to define classes and methods with placeholder.
- The concept of generics is used to create general purpose classes and methods.
- Generics allows us to create a single class or method that can be used with different types of data.
- It helps us to reuse a code.

21] Difference between Array & ArrayList?

- Array has fixed size and can't increase or decrease dynamically while ArrayList Size can be increase or decrease dynamically.
- Arrays are strongly typed which means it can store only specific type of items or elements while ArrayList are not strongly typed.
- In Array Insertion and deletion operation is fast while in ArrayList Insertion and deletion operation is slower than an Array.
- Array cannot accept null while ArrayList can accepts null.

Topic : ASP.NET

1] ASP.net page Life Cycle?

- **Page request:**
When a page is requested by the user, ASP.NET parses and compiles that page.

- **Start :**
In this stage, page properties such as Request and response are set.
- **Initialization**
In this stage, each control's UniqueID property is set.
- **Load :**
During this phase, if page request is postback, control properties are loaded with information.
- **Postback event handling :**
In this stage, event handler is called if page request is postback. After that, the Validate method of all validator controls is called.
- **Rendering :**
Before rendering, view state is saved for the page and all controls. During the rendering stage, the page calls the Render method for each control
- **Unload:**
At this stage the requested page has been fully rendered and is ready to terminate. At this stage all properties are unloaded and cleanup is performed.

2] Explain Global.asax file?

- The Global. asax file is a special file that contains event handlers for ASP.NET application lifecycle events.
- The route table is created during the Application Start event.

3] Difference Between PageLoad & PostBack?

- PageLoad and PostBack are page life cycle event.
- The PageLoad event occurs first and then PostBack
- Post back is a part of page load which can happen only when the page is loaded.

4] Explain Async & Await?

- The async keyword turns a method into an async method, which allows you to use the await keyword in its body.
- When the await keyword is applied, it suspends the calling method and yields control back to its caller until the awaited task is complete.

5] Explain State Management in ASP.net.

There are two types of State management in ASP net. They are :

- Server-side
- Client-side

Server-Side

- Session
- Application
- Cache

Client-Side

- Cookies
- Viewstate
- Control state
- Query String
- Hidden Field

Server-Side of State Management in ASP NET

Session

It is used to store identity and information; information is stored in the server.

Application

This is mainly used to store user activity in server memory and application events.

Cache

This is stored on the server-side, and it is used to implement page caching and data caching. A cache is primarily used to set expiration policies.

Client-Side State Management in ASP.NET

Cookie

It is used to store the session and application information of the user. It can be constant and temporary and works with browser requests.

Control State

It is a technique used to maintain data work in order, properly. We use Control State to use the view state without the possibility of it being disabled by the user.

Hidden Field

This field is used to store values on the client-side. The hidden field works on request and is not displayed on the browser.

Viewstate

It is used to manage page-level state and is used for storing, sending, and receiving information.

Query String

The query string is used to store the value in the URL.

6] Explain Validation Controls in ASP.net.

ASP.NET validation controls validate the user input data to ensure that useless, unauthenticated, or contradictory data don't get stored.

Validation controls:

- **RequiredFieldValidator**

The RequiredFieldValidator control ensures that the required field is not empty.

- **RangeValidator**

The RangeValidator control verifies that the input value falls within a predetermined range.

- **CompareValidator**

CompareValidator control compares a value in one control with a fixed value or a value in another control.

- **RegularExpressionValidator**

The RegularExpressionValidator allows validating the input text by matching against a pattern of a regular expression.

- **CustomValidator**

The CustomValidator control allows writing application specific custom validation routines for both the client side and the server side validation.

7] Explain Caching in ASP.net.

- Caching is a technique of storing frequently used data/information in memory,
- so that, when the same data/information is needed next time, it could be directly retrieved from the memory instead of being generated by the application.
- Caching is extremely important for performance boosting in ASP.NET, as the pages and controls are dynamically generated.

6] Explain Authentication & Authorization in ASP.net.

Authentication is the process of obtaining some sort of credentials from the users and using those credentials to verify the user's identity. Authorization is the process of allowing an authenticated user access to resources. Authentication is always precedes to Authorization;

Authentication providers

- The **windows Authentication** provider lets you authenticates users based on their windows accounts. This provider uses IIS to perform the authentication and then passes the authenticated identity to your code. This is the default provided for ASP.net.
- The **passport authentication** provider uses Microsoft's passport service to authenticate users.
- The **forms authentication** provider uses custom HTML forms to collect authentication information and lets you use your own logic to authenticate users. The user's credentials are stored in a cookie for use during the session.

Selecting an authentication provider is as simple as making an entry in the web.config file for the application

1. <authentication mode="windows">
2. authentication mode="passport">
3. <authentication mode="forms">

Topic : ADO.NET

1] Explain ADO.net.

- ADO.NET is a module of .Net Framework which is used to establish connection between application and data sources.
- Data sources can be such as SQL Server and XML.
- ADO.NET consists of classes that can be used to connect, retrieve, insert and delete data.

ADO.NET Data Providers Objects

Object	Description
Connection	It is used to establish a connection to a specific data source.
Command	It is used to execute queries to perform database operations.
DataReader	It is used to read data from data source. The DbDataReader is a base class for all

	DataReader objects.
DataAdapter	It populates a DataSet and resolves updates with the data source. The base class for all DataAdapter objects is the DbDataAdapter class.

ADO.NET Data Providers Class

Class	Description
SqlConnection	It is used to create SQL Server connection. This class cannot be inherited.
SqlCommand	It is used to execute database queries. This class cannot be inherited.
SqlDataAdapter	It represents a set of data commands and a database connection that are used to fill the DataSet. This class cannot be inherited.
SqlDataReader	It is used to read rows from a SQL Server database. This class cannot be inherited.
SqlException	This class is used to throw SQL exceptions. It throws an exception when an error is occurred. This class cannot be inherited.

2] Difference between DataSet & DataTable.

- DataSet comprises one or many dataset tables which have the in-memory feature.
- DataTable holds a single or unit database table that has an in-memory feature.
- DataSet is formed collectively of datatables.
- DataTable is composed of multiple rows and columns to have better access to data.

3] Difference between ExecuteScalar() & ExecuteReader() & ExecuteNonQuery().

- ExecuteScalar() only returns the value from the first column of the first row of your query.

- ExecuteReader() returns an object that can iterate over the entire result set.
- ExecuteNonQuery() does not return data at all: only the number of rows affected by an insert, update, or delete.

4] Difference between ListView & GridView.

ListView:

- It is Template driven.
- Built-in supports for Data grouping.
- Built-in supports for Insert operation.
- Provides flexible layout to your data.
- Performance is fast is compared to GridView.

GridView:

- It Rendered as Table.
- Need to write custom code for Data grouping.
- Need to write custom code for Insert Operation.
- Performance is slow as compared to ListView.

5] What is ConnectionString in asp.net?

- Connection string holds information that is required to communicate with the database.
- Some of the main parameters present in Asp . Net connection string are:
"name": Name of the connection string.

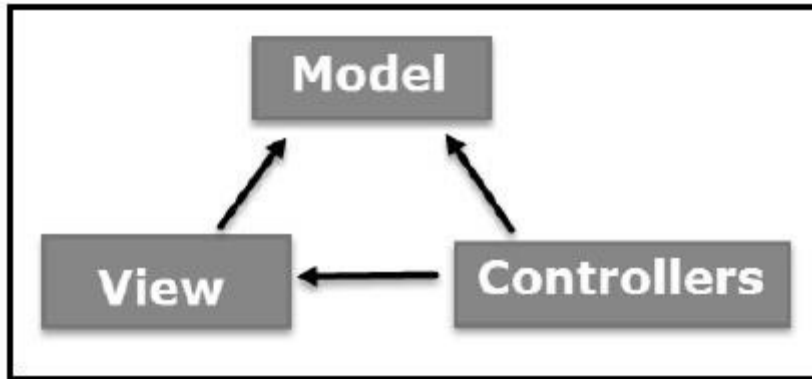
Topic : .NET Core MVC

1] Explain MVC?

- The **Model-View-Controller (MVC)** is an architectural pattern that separates an application into three main logical components: the **model**, the view, and the controller.
- Each of these components are built to handle specific development aspects of an application.
- MVC is one of the most frequently used industry-standard web development framework to create scalable and extensible projects.

MVC Components

Following are the components of MVC –



Model

- The Model component corresponds to all the data-related logic that the user works with.
- This can represent either the data that is being transferred between the View and Controller components or any other business logic-related data.

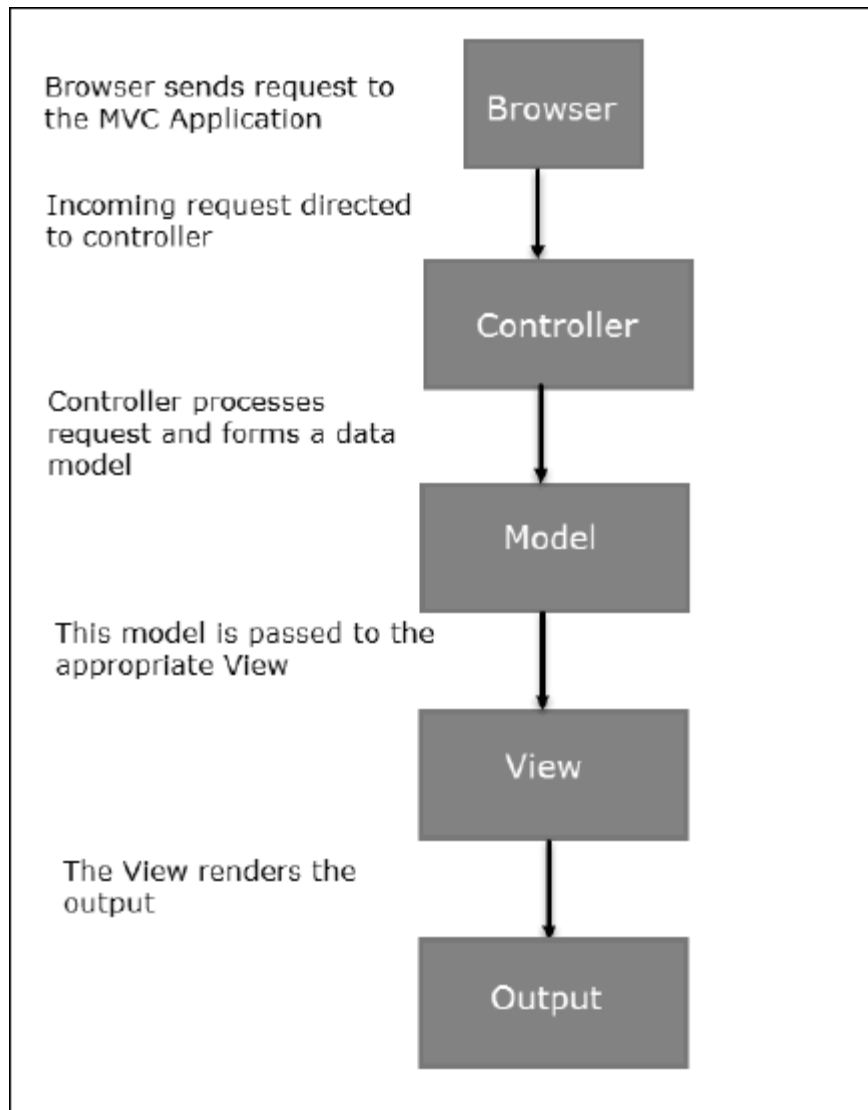
View

- The View component is used for all the UI logic of the application.
- For example, the Customer view will include all the UI components such as text boxes, dropdowns, etc. that the final user interacts with.

Controller

- Controllers act as an interface between Model and View components to process all the business logic and incoming requests, manipulate data using the Model component and interact with the Views to render the final output.
- For example, the Customer controller will handle all the interactions and inputs from the Customer View and update the database using the Customer Model. The same controller will be used to view the Customer data.

2] Explain MVC Flow.



Flow Steps

Step 1 – The client browser sends request to the MVC Application.

Step 2 – Global.ascx receives this request and performs routing based on the URL of the incoming request using the RouteTable, RouteData, UrlRoutingModule and MvcRouteHandler objects.

Step 3 – This routing operation calls the appropriate controller and executes it using the IControllerFactory object and MvcHandler object's Execute method.

Step 4 – The Controller processes the data using Model and invokes the appropriate method using ControllerActionInvoker object

Step 5 – The processed Model is then passed to the View, which in turn renders the final output.

3] Explain Controller Routing.

- In MVC, routing is a process of mapping the browser request to the controller action and return response back.
- Each MVC application has default routing for the default HomeController. We can set custom routing for newly created controller.
- There are 2 Types of Routing:

Conventional routing:

- ASP.net Core MVC application is created by making use of the default template, the app configures a default routing.
- Syntax:

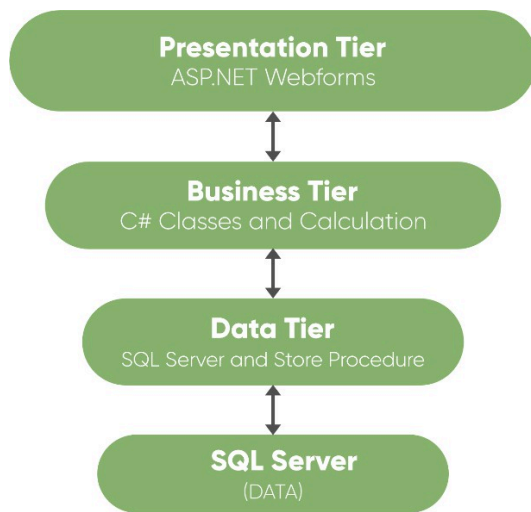
```
app.UseEndpoints(endpoints =>
{
    endpoints.MapControllerRoute
    ( name: "default",
      pattern: "{controller=Home}/{action=Index}/{id?}");
});
```
- use of the **MapControllerRoute()** process for creating a route by providing the name default.

Attribute routing

- possible to use the Attribute Routing feature by placing a route on the action method or the controller.
- Syntax:

```
[Route("")]
[Route("Home")]
[Route("Home/Index")]
public IActionResult
Index()
{
    return View();
}
```

4] Explain N-tier or 3-tier architecture.



i] Presentation Layer :

- This is the top layer of architecture.
- The topmost level of application is the user interface.
- It is related to the user interface that is what the user sees.
- The main function of this layer is to translate tasks and results in something which the user can understand.
- It contains pages like web forms, windows form where data is presented to the user and use to take input from the user.

ii] Business Layer

- This is the middle layer of architecture.
- This layer involves C# classes and logical calculations and operations are performed under this layer.
- It processes the command, makes logical decisions and perform calculations.
- It also acts as a middleware between two surrounded layers that is presentation and data layer.
- This layer also validates the input conditions before calling a method from the data layer.
- This ensures the data input is correct before proceeding, and can often ensure that the outputs are correct as well.
- This validation of input is called business rules.

iii] Data Layer

- This layer is used to connect the business layer to the database or data source.
- It contains methods which are used to perform operations on database like insert, delete, update, etc.
- This layer contains stored procedures which are used to query database. Hence this layer establishes a connection with the database and performs functions on the database

5]Difference between View & Partial View & Strong type

View. View

Views are the general result of a page that results in a display.

Partial View

- A partial view is a Razor markup file (. cshtml) without a @page directive that renders HTML output within another markup file's rendered output.
- Partial view is for a small piece of content that may be reused on different pages.

Strongly type View

- Strongly typed views are used for rendering specific types of model objects, instead of using the general ViewData structure.
- The following list of controls supports the Strongly-Typed HTML Helper. Autocomplete. Checkbox and Radio Button. Numeric, Currency and Percentage Textbox.

6]What is Scaffolding in MVC?

- Scaffolding is a technique used by many MVC frameworks like ASP.NET MVC, to generate code for basic CRUD (create, read, update, and delete) operations against your database effectively.
- Further you can edit or customize this auto generated code according to your need.

7]Why Ajax is used in MVC?

- It is a client-side script that communicates to and from a server/database without the need for a postback or a complete page refresh.
- The Ajax speeds up response time.
- In other words, Ajax is the method of exchanging data with a server, and updating parts of a web page, without reloading the entire page.

8] Explain ViewBag, ViewData, TempData & Session in MVC.(How will you maintain session in MVC?)

ViewBag

- ViewBag is used to pass data from controller to corresponding view.
- If redirection occurs then it's value becomes null.
- ViewBag is slower in comparison to ViewData.
- It is available for the current request.
- It doesn't require typecasting for getting data.

ViewData

- ViewData is a dictionary object that is derived from ViewDataDictionary class.
- ViewData is used to pass data from controller to corresponding view.
- Its life lies only during the current request.
- If redirection occurs then its value becomes null.
- Its required typecasting for getting data and check for null values to avoid error.

TempData

- TempData is a dictionary object that is derived from TempDataDictionary class and stored in short lives session.
- TempData is used to pass data from current request to subsequent request. Its life is very short and lies only till the target view is fully loaded.
- Its required typecasting for getting data and check for null values to avoid error.
- It is used to store only one time messages like error messages, validation messages

Session

- Session is also used to pass data within the ASP.NET MVC application and Unlike TempData,
- It persists for its expiration time. Session is valid for all requests, not for a single redirect.
- It's also required typecasting for getting data and check for null values to avoid error.

9] Difference between keep() & peek().

- The keep() and peek() method is used to read the data without deletion the current read object.
- can use Peek() when you always want to hold/prevent the value for another request.
- can use Keep() when prevent/hold the value depends on additional logic.

10] Explain Model Binding.

- Model Binding is a mechanism that maps the HTTP request data with a model.
- It is the process of creating .NET objects using the data sent by the browser in an HTTP request.
- Model binding is a well-designed bridge between the HTTP request and the C# action methods.

Model Binding Techniques

- **No Binding**

No Binding means access form data directly without binding form to a specific model.

Ex.

```
1. public ActionResult Index()
2. {
3.     ViewBag.Id = Request.Form["Id"];
4.     ViewBag.Name = Request.Form["Name"];
5.     return View("Index");
6. }
```

- **Simple Binding**

In simple binding, pass the parameter in action method with the same name as model properties and MVC Default Binder will automatically map correct action method for the incoming request.

Ex.

```
1. public ActionResult Index(int Id, string Name)
2. {
3.     ViewBag.Id = Id;
4.     ViewBag.Name = Name;
5.     return View("Index");
6. }
```

- **Class Binding**

In Class Binding, pass model as a parameter in action method and then access its entire member variable.

Ex.

```
1. public ActionResult Index(StudentModel sm)
2. {
3.     ViewBag.Id = sm.Id;
4.     ViewBag.Name = sm.Name;
5.     return View("Index");
6. }
```

- **Complex Binding**

Complex Binding means apply multiple related models in a single view Ex.

```
1. [HttpPost]
2. public ActionResult Index(StudentModel sm)
3. {
4.     ViewBag.Id = sm.Id;
5.     ViewBag.Name = sm.Name;
6.     ViewBag.Course = sm.courseModel.Course;
7.     ViewBag.Duration = sm.courseModel.Duration;
8.     return View("Index");
9. }
```

11] Explain Filters in MVC.

- MVC Filter is a custom class where you can write custom logic to execute before or after an action method executes.
- Filters can be applied to an action method or controller in a declarative or programmatic way.
- Declarative means by applying a filter attribute to an action method or controller class and programmatic means by implementing a corresponding interface

Filter Type	Description	Built-in Filter	Interface
Authorization filters	Performs authentication and authorizes before executing an action method.	[Authorize], [RequireHttps]	IAuthorizationFilter
Action filters	Performs some operation before and after an action method executes.		IActionFilter
Result filters	Performs some operation before or after the execution of the view.	[OutputCache]	IResultFilter
Exception filters	Performs some operation if there is an unhandled exception thrown during the execution of the ASP.NET MVC pipeline.	[HandleError]	IExceptionHandler

12] Explain Bundling & Minification.

- Bundling and minification techniques were introduced in MVC 4 to improve request load time.
- Bundling allows us to load the bunch of static files from the server in a single HTTP request.
- Minification technique optimizes script or CSS file size by removing unnecessary white space and comments and shortening variable names to one character.

Bundle Types

ScriptBundle: ScriptBundle is responsible for JavaScript minification of single or multiple script files.

StyleBundle: StyleBundle is responsible for CSS minification of single or multiple style sheet files.

DynamicFolderBundle: Represents a Bundle object that ASP.NET creates from a folder that contains files of the same type.

13] Explain Action Method.

- Action method must be public.
- It cannot be private or protected. Action method cannot be overloaded.
- Action method cannot be a static method.
- Every controller can have a default action method as per the configured route in the `RouteConfig` class.
- By default, the `Index()` method is a default action method for any controller, as per configured default root.

Result Class	Description
ViewResult	Represents HTML and markup.
EmptyResult	Represents No response.
ContentResult	Represents string literal.
FileContentResult/FilePathResult/FileStreamResult	Represents the content of a file.
JsonResult	Represent JSON that can be used in AJAX.
RedirectResult	Represents a redirection to a new URL.
RedirectToRouteResult	Represent another action of same or other controller.
PartialViewResult	Returns HTML from Partial view.

13] Explain Action Verbs in MVC.

- Action Verbs selector is to handle different type of Http requests.
- The MVC framework includes HttpGet, HttpPost, HttpPut, HttpDelete, HttpOptions, and HttpPatch action verbs.
- If you don't apply any action verbs to an action method, then it will handle HttpGet request by default.

Http method	Usage
GET	To retrieve the information from the server. Parameters will be appended in the query string.
POST	To create a new resource.
PUT	To update an existing resource.
HEAD	Identical to GET except that server do not return the message body.
OPTIONS	It represents a request for information about the communication options supported by the web server.
DELETE	To delete an existing resource.
PATCH	To full or partial update the resource.

Topic : Entity Framework

1] Explain Entity Framework.

- Entity Framework was first released in 2008, Microsoft's primary means of interacting between .NET applications and relational databases.
- Entity Framework is an Object Relational Mapper (ORM) which is a type of tool that simplifies mapping between objects in your software to the tables and columns of a relational database.
- Entity Framework (EF) is an open source ORM framework for ADO.NET which is a part of .NET Framework.

DbContext

- DbContext instance represents a combination of the Unit of Work and Repository patterns such that it can be used to query from a database and group together changes that will then be written back to the store as a unit.
- The context class manages the entity objects during run time, which includes populating objects with data from a database, change tracking, and persisting data to the database
- DbContext corresponds database.

DbSet

- The DbSet class represents an entity set that can be used for create, read, update, and delete operations.
- DbSet corresponds to a table or view in your database.

2] Explain Entity Framework Approaches.

- Code First
- Database First
- Model First

Code First

- Code First modeling workflow targets a database that doesn't exist and Code First will create it.
- It can also be used if you have an empty database and then Code First will add new tables too.
- Code First allows you to define your model using C# classes.

Model First

- Model First is great for when you're starting a new project where the database doesn't even exist yet.
- The model is stored in an EDMX file and can be viewed and edited in the Entity Framework Designer.
- In Model First, you define your model in an Entity Framework designer then generate SQL, which will create database schema to match your model and then you execute the SQL to create the schema in your database.
- The classes that you interact with in your application are automatically generated from the EDMX file.

Database First

- The Database First Approach provides an alternative to the Code First and Model First approaches to the Entity Data Model.
- It creates model codes (classes, properties, DbContext etc.) from the database in the project and those classes become the link between the database and controller.
- The Database First Approach creates the entity framework from an existing database.

- We use all other functionalities, such as the model/database sync and the code generation, in the same way we used them in the Model First approach.

3] Explain Eager Loading & Lazy Loading.

Eager Loading

- Eager loading is the process whereby a query for one type of entity also loads related entities as part of the query.
- Eager loading is achieved by the use of the **Include method**.
- It means that requesting related data be returned along with query results from the database.
- There is only one connection made to the data source, a larger amount of data is returned in the initial query.
- For example, when querying students, eager-load their enrollments. The students and their enrollments will be retrieved in a single query.

Lazy Loading

- Lazy loading is the process whereby an entity or collection of entities is automatically loaded from the database the first time that a property referring to the entity/entities is accessed.
- Lazy loading means delaying the loading of related data, until you specifically request for it.
- For example, when using the Student entity class, the related Enrollments will be loaded the first time the Enrollments navigation property is accessed

4] Explain Data Annotation.

- DataAnnotations is used to configure the classes which will highlight the most commonly needed configurations.
- allows these applications to leverage the same annotations for client-side validations.
- DataAnnotation attributes override default CodeFirst conventions.

System.ComponentModel.DataAnnotations includes the following attributes that impacts the nullability or size of the column.

- Key
- Timestamp
- Required
- MinLength
- MaxLength
- StringLength

System.ComponentModel.DataAnnotations.Schema namespace includes the following attributes that impacts the schema of the database.

- Table
- Column
- Index
- ForeignKey

Topic : Web API

1]What is API?

- API is some kind of interface which has a set of functions that allow programmers to access specific features or data of an application, operating system or other services.
- Web API as the name suggests, is an API over the web which can be accessed using HTTP protocol.

2]What is Web API?

- Web API is an extensible framework for building HTTP based services that can be accessed in different applications on different platforms such as web, windows, mobile etc.
- It works more or less the same way as ASP.NET MVC web application except that it sends data as a response instead of html view.
- It is like a webservice or WCF service but the exception is that it only supports HTTP protocol.
- Web API is an ideal platform for building RESTful services.

Http method	Usage
GET	To retrieve the information from the server. Parameters will be appended in the query string.
POST	To create a new resource.
PUT	To update an existing resource.
HEAD	Identical to GET except that server do not return the message body.
OPTIONS	It represents a request for information about the communication options supported by the web server.
DELETE	To delete an existing resource.

Http method	Usage
PATCH	To full or partial update the resource.

3]Explain Web API Controllers?

It handles incoming HTTP requests and send response back to the caller. Controller class must be derived from **ApiController** class.

4]Explain Web API Filters?

Web API includes filters to add extra logic before or after action method executes. Filters can be used to provide cross-cutting features such as logging, exception handling, performance measurement, authentication and authorization.

Filter Type	Interface	Class	Description
Simple Filter	IFilter	-	Defines the methods that are used in a filter
Action Filter	IActionFilter	ActionFilterAttribute	Used to add extra logic before or after action methods execute.
Authentication Filter	IAuthenticationFilter	-	Used to force users or clients to be authenticated before action methods execute.
Authorization Filter	IAuthorizationFilter	AuthorizationFilterAttribute	Used to restrict access to action methods to specific users or groups.
Exception Filter	IExceptionFilter	ExceptionFilterAttribute	Used to handle all unhandled exception in Web API.

Filter Type	Interface	Class	Description
Override Filter	IOverrideFilter	-	Used to customize the behaviour of other filter for individual action method.

5] Explain Dependency Injection(DI)?

- Dependency injection is used to make a class independent of its dependencies or to create a loosely coupled program.
- Dependency injection is useful for improving the reusability of code.
- Likewise, by decoupling the usage of an object, more dependencies can be replaced without needing to change class.
- It is an implementation of the Inversion of Control (IoC) principle

6] Explain Web API Hosting?

The Web API application can be hosted in two ways.

- IIS Hosting
- Self Hosting

IIS Hosting

- Web API can be hosted under IIS, in the same way as a web application.
- So, when you host your MVC web application under IIS it will also host Web API that uses the same base address.

Self Hosting

- You can host a Web API as separate process than ASP.NET.
- It means you can host a Web API in console application or windows service or OWIN or any other process that is managed by .NET framework.

You need to do following steps in order to self-host a web API.

1. Use `HttpConfiguration` to configure a Web API
2. Create `HttpServer` and start listening to incoming http requests

SQL

1. Difference between primary & unique key.

- The primary key uniquely identifies each record in the table.
- The unique key serves as a unique identifier for records when a primary key is absent.
- The primary key cannot store NULL values.
- The unique key can store a null value, but only one NULL value is allowed.

2. Difference between primary & foreign key.

- A primary key generally focuses on the uniqueness of the table.
- It assures the value in the specific column is unique.
- A foreign key is generally used to build a relationship between the two tables.
- The table allows only one primary key.

3. What are the Constraints in SQL?

SQL constraints are used to specify rules for the data in a table.

Constraints are used to limit the type of data that can go into a table.

The following constraints are commonly used in SQL:

- [NOT NULL](#) - Ensures that a column cannot have a NULL value
- [UNIQUE](#) - Ensures that all values in a column are different
- [PRIMARY KEY](#) - A combination of a [NOT NULL](#) and [UNIQUE](#). Uniquely identifies each row in a table
- [FOREIGN KEY](#) - Prevents actions that would destroy links between tables
- [CHECK](#) - Ensures that the values in a column satisfies a specific condition
- [DEFAULT](#) - Sets a default value for a column if no value is specified
- [CREATE INDEX](#) - Used to create and retrieve data from the database very quickly

4. Explain Indexes in SQL?

- Indexes are used to retrieve data from the database more quickly than otherwise.
- The users cannot see the indexes, they are just used to speed up searches/queries.

CREATE INDEX Syntax

```
CREATE INDEX index_name
ON table_name (column1, column2, ...);
```

